Assignment 8, explained

A neural network that reads handwritten digits

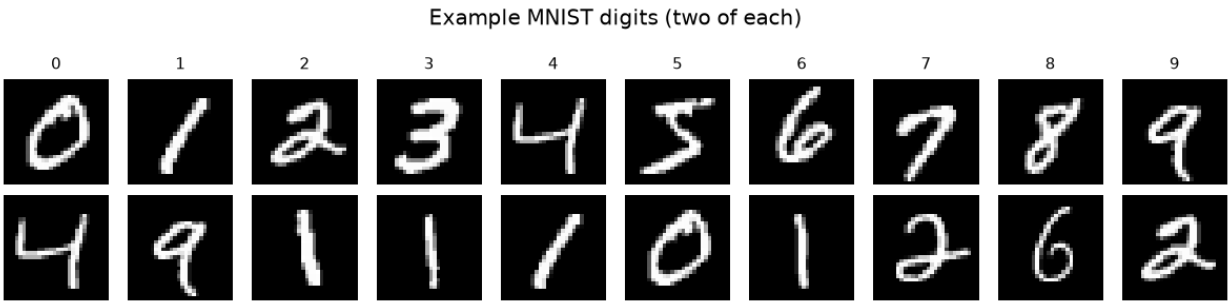
A plain-language walkthrough of the MNIST network,
with the numbers from the solution notebook.

SVNIT · Department of Artificial Intelligence
Machine Learning · u24ai028

What you are asked to build

MNIST is 70,000 pictures of handwritten digits, already split into 60,000 for training and 10,000 for testing. Each picture is 28 by 28 pixels, grayscale, and the label is the digit 0–9.

The model is a feed-forward network: an input layer, hidden layers, and a softmax output. It is trained with cross-entropy loss and Adam. This lab environment does not have TensorFlow or PyTorch, so the notebook uses sklearn's MLPClassifier. That class is the same kind of network: 784 inputs, ReLU hidden units, a 10-way softmax, cross-entropy, and solver='adam'.



Two examples of each digit. The same digit is written differently every time, which is why a straight classifier is not enough.

Preprocessing

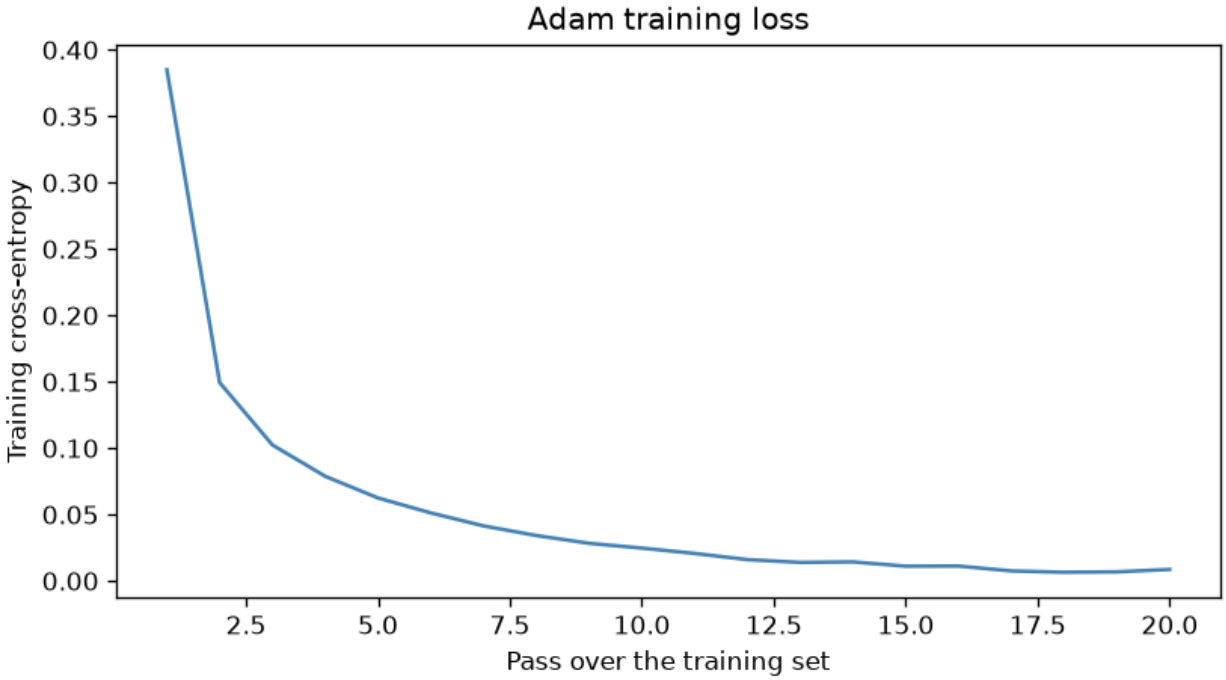
- Pixels arrive as integers from 0 to 255. Divide by 255 so every input is between 0 and 1. Large inputs make the first weight updates too violent.
- Flatten each 28×28 picture into a vector of 784 numbers. An ordinary neural net has no notion of 'the pixel above'. A convolutional net would keep the grid; this assignment asks for an ANN.
- The official 60,000 / 10,000 split is kept. Reshuffling it would make the test score incomparable with published MNIST numbers.

The ten digits are roughly balanced in the training set (about 5,400 to 6,700 images each), so accuracy is not the trap it was on the fraud data.

The network, layer by layer

- Input, 784 units. One unit per pixel. This layer stores the image. It does not learn weights of its own.
- Hidden layer, 128 units. Each unit computes ReLU(w · x + b), and ReLU is max(0, z). Stacking these lets the net bend, which logistic regression cannot do.
- Hidden layer, 64 units. A second ReLU layer, reading the 128 units rather than the raw pixels.
- Output, 10 units, then softmax. Softmax turns the 10 scores into probabilities that add up to 1. The predicted digit is the largest probability.

Cross-entropy is the loss. If the true digit is 7 and the network says 7 with probability 0.9, the loss is small. If it says 7 with probability 0.01, the loss is large. Adam updates each weight with its own step size, based on how noisy that weight's gradient has been. The starting step size used here is 0.001. The network sees the training set 20 times. Training cross-entropy ends at 0.0086.



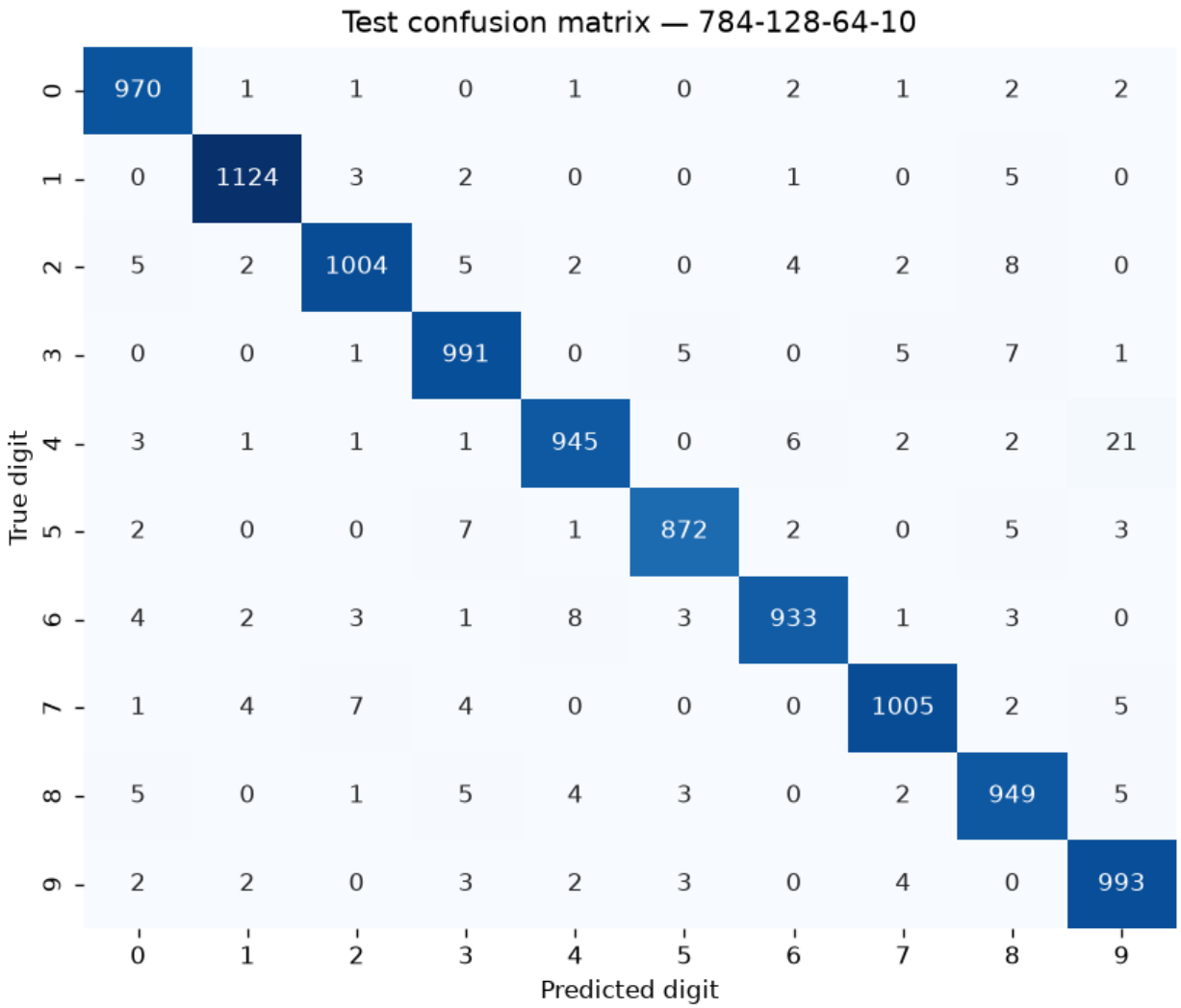
The loss falls quickly and then flattens. Twenty passes are enough for this size of network.

Test scores

On the 10,000 images the network never trained on:

Metric	Score	Meaning
Accuracy	97.86%	fraction of digits read correctly
Precision	0.979	macro: each digit weighted the same
Recall	0.978	macro: each digit weighted the same

Digit 1 is the easiest (precision and recall both about 0.99). Digits 4, 8, and 9 are the ones it mixes: a 4 can look like a 9, and an 8 can look like a 3 or a 5. The confusion matrix is almost diagonal, which is what a good digit reader looks like.



Rows are the true digit, columns are the prediction. Off-diagonal cells are the mistakes.

Say this: The 784-128-64-10 network reads 97.86% of the test digits. Precision and recall are essentially the same number because the classes are balanced.

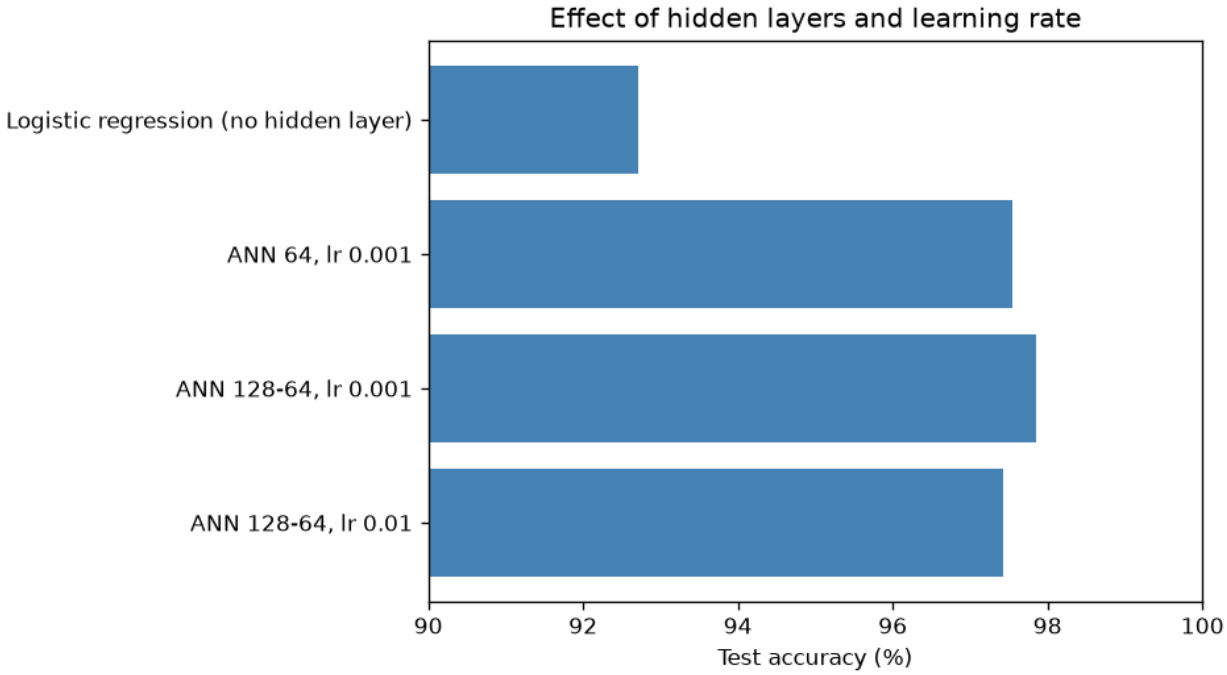
Hyperparameters, against a plain benchmark

One change at a time. Logistic regression is the same softmax with no hidden layer, so it shows what the hidden layers are worth.

Model	Accuracy	Precision	Recall
Logistic regression	92.71%	0.926	0.926
ANN, 64 units, lr 0.001	97.54%	0.975	0.975
ANN, 128-64, lr 0.001	97.86%	0.979	0.978
ANN, 128-64, lr 0.01	97.42%	0.974	0.974

- Adding even one hidden layer jumps accuracy from 92.7% to 97.5%. A digit is not a straight cut through pixel space.
- A second hidden layer adds a bit more, 97.54% to 97.86%. Extra capacity helps, then the gains get small.
- Raising Adam's step from 0.001 to 0.01 lowers accuracy to 97.42%. The network still learns, but the steps are too coarse and it settles in a worse place.

Published context, not trained in this notebook: a linear classifier on MNIST is usually about 92%, which matches the logistic regression here. A fully connected network of this size is usually 97–98%, which matches this ANN. A convolutional network, which knows that neighbouring pixels form strokes, is usually above 99%. This network cannot use that, because the picture was flattened.



The axis starts at 90% so the gap between logistic regression and the networks is visible.

Say this: Hidden layers are worth about 5 points over logistic regression. Two layers and a learning rate of 0.001 is the best of the four settings I tried, at 97.86%.